

Biofilm2 – Group 5

Project leader: Ruth Rudolph

RDLRUT001

Byron Clarke

CLRBYS003

Sonali Naidoo

NDXSON001

Abstract	-----
Introduction	-----
Requirements	-----
Design	-----
Implementation	-----
Program validation and verification	-----
Conclusion	-----
References	-----
Appendices	-----
A: User manual	-----
B: Test results	-----
C: How work was delegated between members	-----
D: Declaration of generative AI uses	-----

Abstract:

The project outlined below consists of a multithreaded Java program which simulates early-stage biofilm formation. The system models a growing colony of rod-shaped bacteria with the ability to move, reproduce, secrete extracellular polymeric substances (EPS), and take in nutrients on a two-dimensional grid. The project is modular, consisting of three core sections; firstly, a Java-based multithreaded simulation module which accepts an input file containing simulation parameters, performs the simulation and outputs results to an output file, allowing users to rerun visual displays or produce various graphs without rerunning the simulation. The output file contains details including particle positions over time as well as quantifying variables including sparseness and roughness. The particle position data is used by the second Java-based program, the visualization module which produces a movie-like representation of the colony over time. Finally, the python-based graphing module allows the user to critically examine several aspects of the simulation, by producing various graphs based on the

simulation output. The three modules together form a program which allows for thorough investigation of colony behaviour under various settings, making it a potentially useful basis from which to expand and investigate biofilm formation in more detail.

Introduction:

The aim of this project is to develop a particle-based biophysical model that simulates early-stage biofilm formation, with focus centered on the role of extracellular polymeric substances (EPS) in driving cell aggregation. The model will represent a growing and expanding bacterial colony of mobile, rod-shaped bacteria which can move, reproduce, and secrete EPS. The primary goal is to explore how secreted EPS impacts the clustering of bacterial cells into dense, aggregated structures, which are an important precursor to mature biofilms. The simulation will be implemented in Java as a multithreaded application. It will include a simulation module, a separate visualization module for presentation of the results, and a graphing module to present graphs of the results. The model will serve as a research tool for understanding the underlying biophysical mechanisms of biofilm formation, as well as a framework that can potentially be extended to incorporate more complex biological processes. The implications of this work extend beyond a single research group. Biofilms have a significant impact and create challenges in both medical and industrial contexts, making a broad community of researchers, clinicians, and engineers indirect stakeholders. In healthcare, biofilms are associated with persistent infections such as chronic wounds and dental plaque, and they frequently colonize medical implants—leading to severe complications and increased treatment costs. In industry, biofilms contribute to food contamination, reduce equipment lifespan, and create inefficiencies in water treatment systems. As such, this project may be valuable to both academic and applied research communities seeking insight into early-stage biofilm formation.

Our approach was primarily an evolutionary prototype driven approach. We did an initial analysis and design of our project in the first few weeks and then would iteratively create and test prototypes with additional features on each cycle. These cycles were asynchronous, and not formally implemented, hence this is something between a waterfall-based approach and a proper agile approach, since there was only one main analysis and design window, but many iterations of the project.

This simulation was designed to mimic the paper Bera et al. (2023) Our aim was to reproduce their results by following their methods. Another of the aims is to determine if their results are correct for the methods they provided. They did not have a code review in their paper, which is the primary reason it is not trivial to replicate. Their methods also were often ambiguous and did not fully explain many aspects of the calculations and implementations they used in their paper. Regardless this is important as we did not need to research the topic in detail to construct our model's design.

Requirements:

The requirements for this system focus mainly on useability and consistency, with no formal requirements for performance but rather an aim for generally reasonable run times and visual display. The system is designed to be used primarily by academics and thus has limited use

cases due to the niche nature of the software. Outlined below are the functional requirements in the form of use cases

Use Case 1 – Run Biofilm Simulation

Goal: Generate a simulated model of early-stage biofilm growth using defined input parameters.

Preconditions:

- Simulation parameters have been correctly entered into an input file
- Simulation engine is operational.

Use:

- Researcher enters the name of the input file and output file into the console or when starting the program via a makefile.
- Researcher begins the simulation
- Simulation module:
 - Validates inputs
 - Runs calculations for bacterial movement, growth, reproduction and EPS secretion
 - Outputs simulation data to the output file

Alternative paths:

- If input file is not found, system alerts user and allows them to try again.
- If input file contains invalid data, system alerts user and allows them to try again.
- If parameters are too large, system stops the simulation and alerts user.
- Simulation runs until end point is reached

Use Case 2 – View Simulation Results

Goal: Visualise the simulation which has been run, via the output file produced by the simulation module.

Preconditions:

- Simulation has been run and an output file produced.
- The output file is unedited and thus is of the correct format with valid data.
- User has started the visualisation module and been directed to an options menu.
- Visualization engine is operational.

Use:

- User selects/enters visualisation parameters at the options menu
- User clicks View simulation
- Visualization module processes output into a video-like simulation over time (faster than true time)
- Researcher can:
 - Pause at any time during the visualisation
 - Click to re-watch the simulation once completed
 - Return to the settings menu once completed

Alternative paths:

- Different visualisation options will lead to different outputs.
 - Adjust the frame rate (delay between frames)
 - Hide eps or bacteria
 - Change eps or bacteria colour
 - Show eps or bacteria clusters
 - Change background colour
 - Manual click-through option
 - Artificial size increase for eps and/or bacteria
- User may pause or exit the visualisation at any time
- Manual click through option:
 - User must click next frame to proceed at each frame rather than automatically being shown the frames after a given delay period

Use Case 3 – Generate Simulation Graphs

Goal: Visualise the simulation data in the form of various graphs

Preconditions:

- Simulation has been run and various output files produced.
- User has started the graphing module
- Graphing engine is operational.

Use:

- User enters the names output files to use
- User selects which graph to view
- Visualization module processes output into various graphs based on user selection
- User Views the selected graph

Alternative paths:

- User can choose whether to load files by manually typing them or through the file browser
- Different graphing options will lead to different output graphs
- User may exit the graphing section at any time
- User may switch between graphs
- User can view graphing data for debugging purposes

Non-functional requirements:

The system should allow for a simulation to run in reasonable time through the use of multithreading. The system should also be reasonably reliable, useable and maintainable.

Design:

Interaction between modules

Overall, the system follows this small pipeline between the 3 modules. The separation into 3 programs allows the user to run a simulation once, which will take the most time, and then they can visualize and graph different aspects of the simulation to produce the different figures seen in Bera et al. (2023) The graphing and visualization were separated as they serve entirely different functions: the graphing produces single graphs that display different results and summaries of the simulation, while the visualization produces a playback of the simulation that can be watched, paused, and stepped through frame by frame.

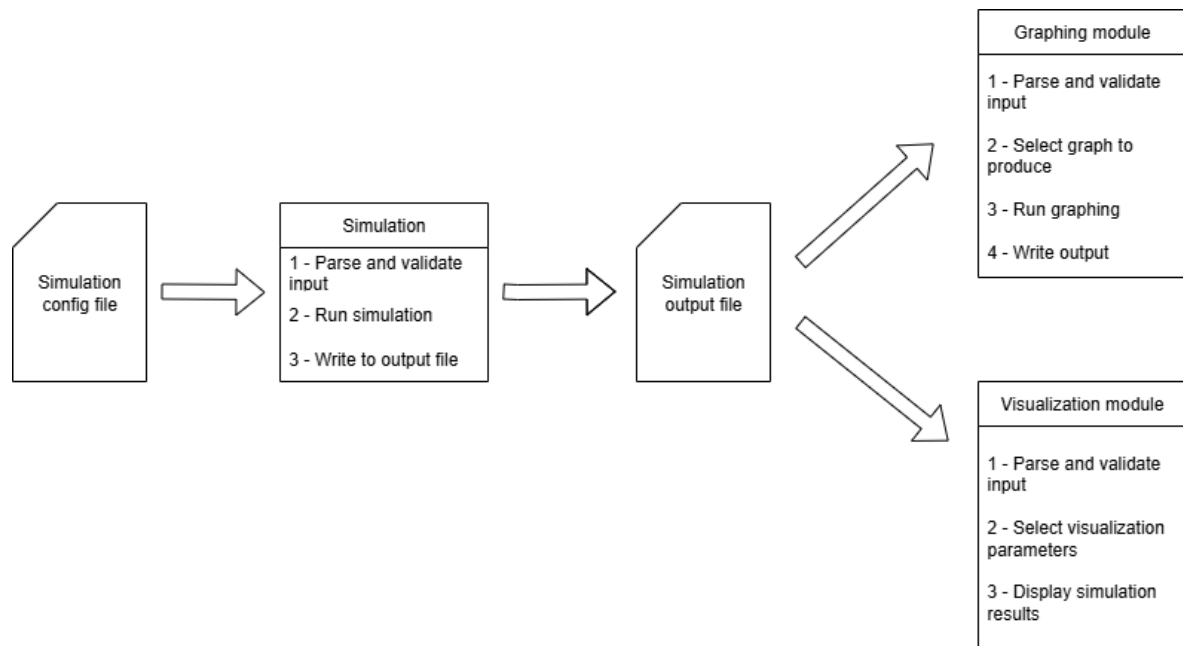
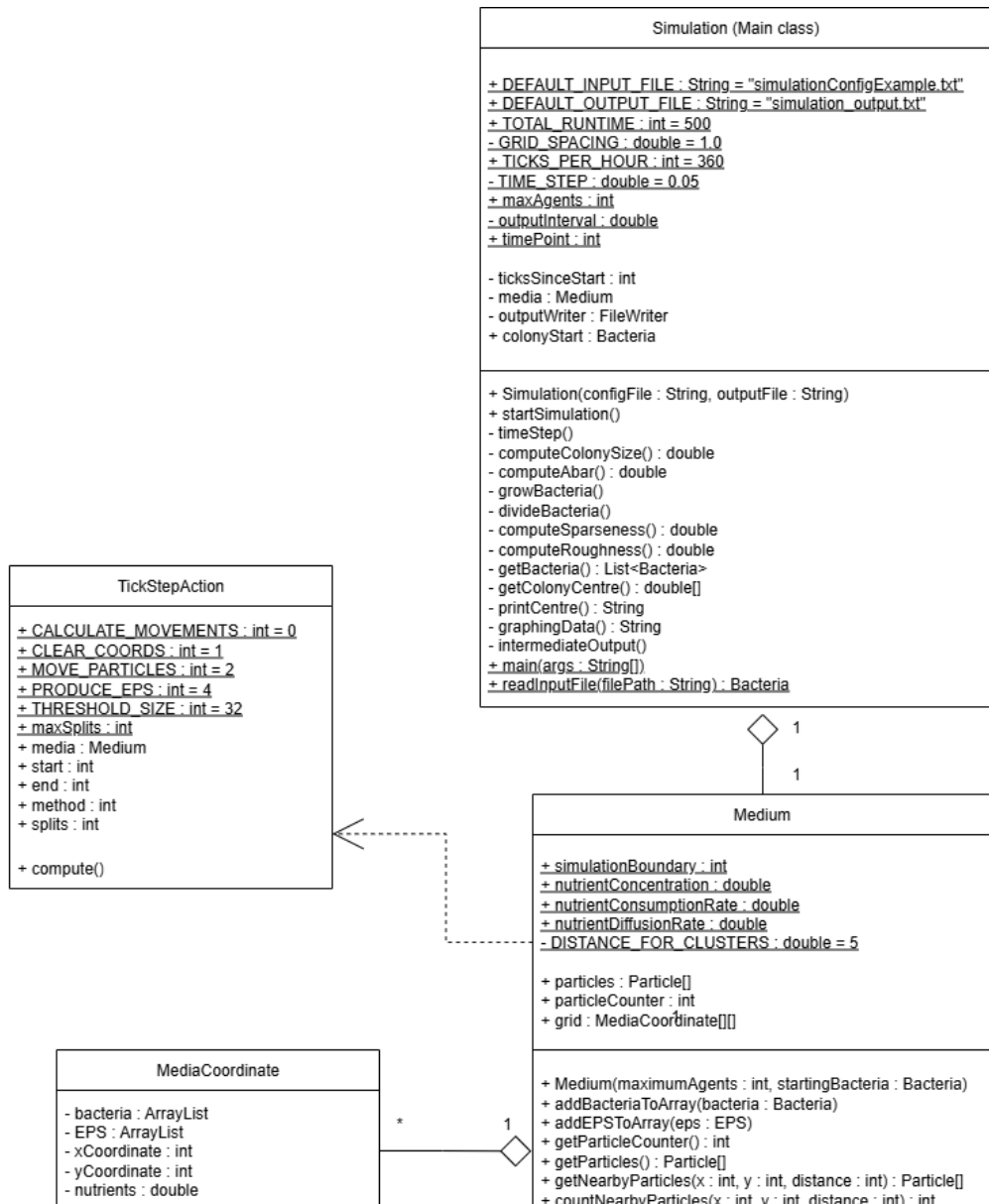


Fig. 1: Overall architecture for the system, in terms of information flow from the user's perspective. User creates a config file with simulation parameters, then runs the simulation which produces an output file. Finally, the user inputs that file to the visualization or graphing module to see results.

Simulation:



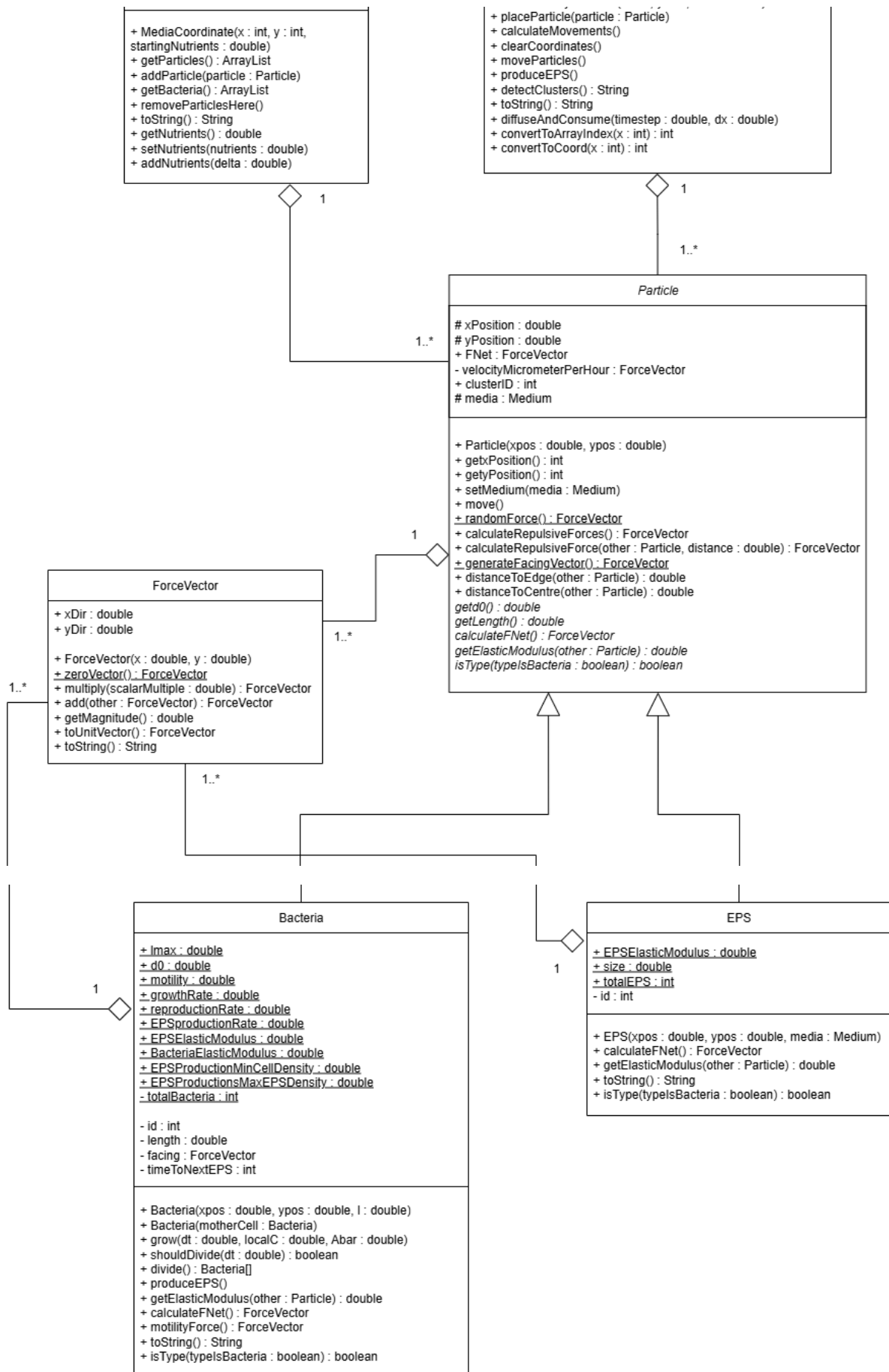


Fig. 2: UML diagram of simulation module. There may be some occasional disparity between this and the actual program, but never significant. Typically there are more variables in the actual program that are public due to unit tests needing access to them.

In this module, the simulation class creates a Medium instance and then calls methods of other classes to run the simulation in order and write output to the output file. Medium represents the physical 2D space the bacteria live in. It is subdivided into a grid of MediaCoordinates that each represent $1\mu\text{m}^2$ of the space and can store many particles. Medium contains many methods that are called to process the different calculations for the particles, which then call methods in individual particles.

The primary data structures in the simulation are the Particle array (Particles) in Medium, and the MediaCoordinate matrix (Grid) in Medium. Having these 2 structures is ideal as when all particles need to be updated by a method it is much quicker to traverse a small array, than to check every grid cell for particles. It also makes finding particles close to one being processed fast as only a few nearby grid squares need to be checked.

Simulation algorithms:

getNearbyParticles, and countNearbyParticles are vital helper methods. They search for particles in all grid squares in a square centred on the coordinate given. They are ideal for finding nearby particles as they can be set to only look for particles in the area right around the coordinate given. This saves particles from having to check the position of every other particle for various calculations.

calculateMovements, clearCoordinates, moveParticles, and produceEPS are all similar. In each case all particles must be processed before the next step, but within each step a forkJoin framework is used to parallelize the operations. They traverse through the particle array to access all the particles and then process each in turn. This separation into many methods that run in sequence each simulation tick allows for decent parallelism, and keeps code organised.

calculateRepulsiveForces is a method in each particle that is called by calculate movements. The particle array is used to make each particle call this method and then each particle uses the grid to check nearby grid squares for particles that cause a repulsive force. This method integrates the use of both data structures to minimize processing time at every opportunity.

detectClusters repeatedly adds nearby bacteria or eps to a cluster until there are no new particles near to the group and then processes a new cluster if there are still particles that aren't part of a cluster. This algorithm repeatedly uses getNearbyParticles to avoid every particle comparing to every other particle.

diffuseAndConsume computes nutrient diffusion across the grid using DiffuseTask, which is parallelized by using a forkJoin framework. This creates a temporary nutrient array, which is swapped with the main nutrient array. After diffusion, the method iterates over only grid cells that are occupied by bacteria and implements nutrient consumption (based on a Monod uptake).

computeAbar calculates the average effective surface area of all bacteria by computing the individual area of each bacterium based on given area equations. The method then sums the areas and divides them by the total number of bacteria present, returning the average area.

growBacteria simulates the growth of all bacteria in a single time step. It computes calls computeAbar, and then for each bacterium, retrieves the nutrient concentration of the cell the bacterium is in. It then calls the grow method, which uses these values to update the bacteriums size according to the growth rate parameter, effectively updating the length of all bacteria in the colony.

divideBacteria checks whether each cell meets the division criteria speicfied in shouldDivide, and if found true, calls the divide method, which returns two daughter cells. After all the bacteria have been checked, the daughter cells are aded to the particle array and to their repsective cells in the media.

ComputeSparseness and computeRoughness quantify the spatial distribution of the bacteria. ComputeSparseness finds the bounding rectangle of all the bacterial positions and compares the number of occupied cells to the total area, while computeRoughness measures the irregularity of the colony's shape by computing the distances of all bacteria from the centre and computing the standard deviation of these distances.

Visualisation:

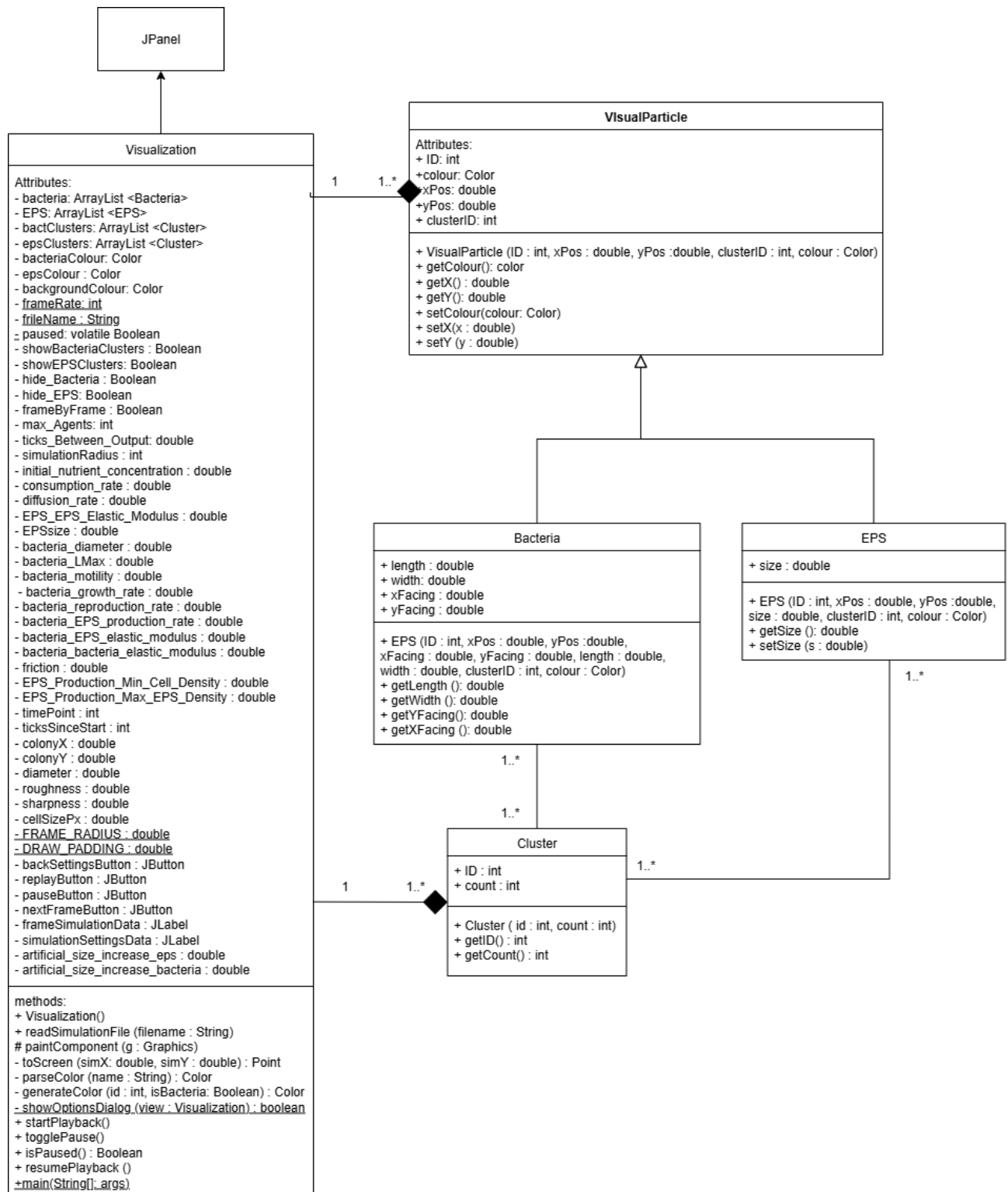


Fig. 3: UML diagram of the visualization module.

This module is designed to accept a file which contains the data produced by the simulation module, process the data and display a visual representation thereof, to allow the user a visual representation of the simulation results. The program starts by providing the user with a set of parameters which can be set according to the needs of the user and how they would like to view the simulation. Once the parameters for the visualisation have been chosen by the user and the user starts the visualization, the module begins by reading and storing the input

simulation parameters from the file, and thereafter reading and storing the bacteria, eps, bacteria cluster and eps cluster data as array Lists of objects. Thereafter, it uses the PaintComponent method to display each frame (time point) on a Jpanel, wait for a period and then read in and display the next frame.

The ArrayLists form the main data structures of the visualization module, there are 4 ArrayLists consisting of 3 different object types, Bacteria, EPS and 2 cluster arrays; one for bacteria clusters and the other for eps clusters. The arrayLists are used to allow a dynamic number of objects to be added and changed not only between simulation files but also between time points as the bacteria reproduce. These arrayLists are then used in the PaintComponent method to draw out the simulation thus reducing the amount of information passed between methods as they are global objects as well as enabling easy storage and grouping of information.

Visualization algorithms:

readSimulationFile is the main processing method of this class and is where the file data is read and placed into objects, it also controls the frame rate by pausing the thread for the given period or (in the case of manual frame click through or pause button selection) until signalled by the user. This method also calls the PaintComponent method in each iteration to load the data onto the frame where the user can view it.

showOptionsDialog displays the options menu for simulation settings to the user, validates the inputs and then processes the information. It returns a Boolean, true if the user selects to start the simulation or false if they choose to exit.

toScreen is a helper method which translates bacteria grid positions to pixel values based on the frame size and simulation boundary. It was created because the bacteria and eps both require the same translation.

parseColor is a helper method which translates the colours selected in the options menu (Strings) by the user to Color objects. The helper method was created so that it can be used for the bacteria, eps and background colour settings.

generateColor is a helper method which uses cluster ID as well as the number of particles in the cluster to set the colour/shade of all the elements in the cluster when the user has selected to view the clusters. It is combined with the maps to allow flexibility in the colour generation based on the cluster sizes.

startPlayback, togglePause, isPaused and resumePlayback all perform simple functions in relation to the display, as per their names. They control the start, pausing and resuming of the visual display when called.

Implementation:

Data structures:

The grid (grid in Media.java) is a matrix of MediaCoordinates. A MediaCoordinate represents $1\mu\text{m}^2$ and can store multiple bacteria or eps. A particle (bacteria or eps)'s x and y position defines where in the grid it is stored.

The particle array (particles in Media.java) stores all particles present in the simulation. It provides a faster alternative to searching through all the grid cells when every particle needs to be accessed for an operation, which happens frequently.

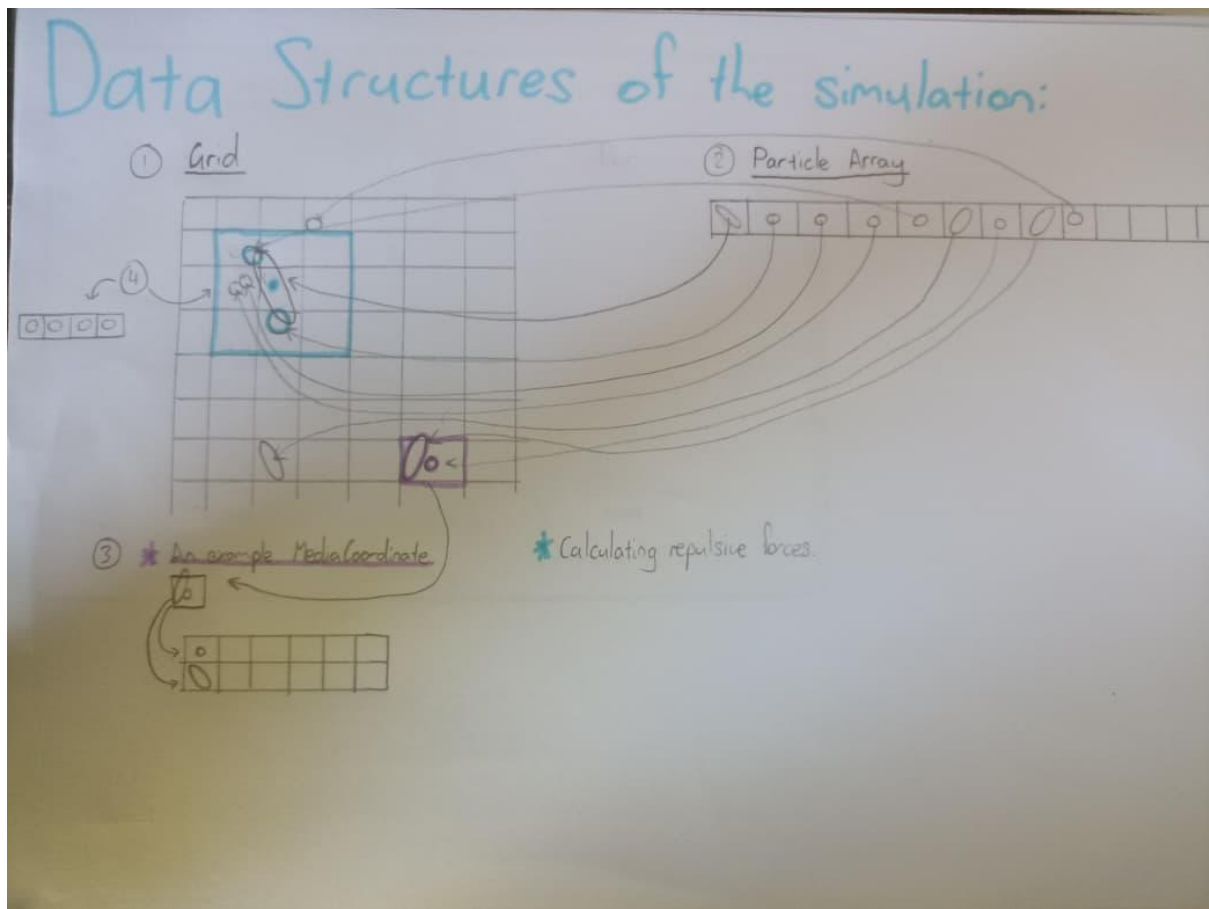


Fig. 4: Diagram explaining the grid and particle array. There are 4 items pointed out. 1: The grid itself is a 2D array of MediaCoordinates containing particles. They are placed in the grid according to their position. 2: The particle array has references to every particle for easy access to all particles quickly, since there are many empty spaces on the grid which would make using that inefficient. 3: An example MediaCoordinate, this one has 2 particles, and we see it holds them in a bacteria and a eps array which can be retrieved when necessary. 4: a repulsive force calculation for the bacterium. All particles in the blue square are added to a list. The one outside of this area is ignored since it will not be close enough to possibly exert repulsive forces on this bacterium. Two of those EPS overlap with the bacterium and therefore apply a repulsion.

Visualization:

Visualization.java has 4 main data structures used, each of which is an ArrayList of objects.

ArrayList<Bacteria> stores all bacteria objects for the given simulation and is updated with every time point. It makes use of direct indexing through the ID values of the bacteria. The use of an ArrayList allows for dynamic growth between time points and simulations.

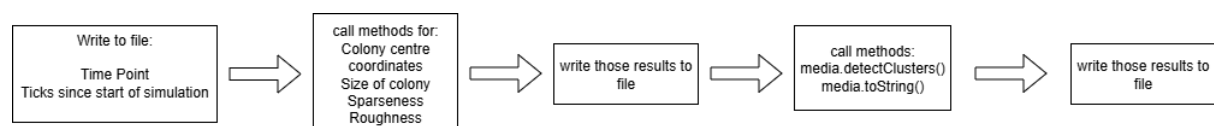
ArrayList<EPS> stores all EPS objects for the given simulation and operates as the bacteria arrayList does.

ArrayList<Cluster>; there are 2 such ArrayLists, one for Bacteria clusters and another for EPS clusters, each operates as the Bacteria and EPS ArrayLists do. These are also used by the generateColor method, to allocate colours to the bacteria and eps according to cluster size.

Significant methods:

Simulation.java:

- readInputFile(): This method reads in the given input file and assigns all the simulation parameters which are typically static variables in the various classes. It then returns a bacterium to start the colony.
- timeStep(): This method calls all the necessary procedures that need to be done at each simulation tick, in order:
calculateMovements(), clearCoordinates(), moveParticles(), diffuseAndConsume(), growBacteria(), divideBacteria()
- detectClusters(): This method calculates all the bacteria and EPS clusters at the present point in time, so that they can be added to the output file.
- growBacteria(): This method calls Bacteria.grow() for each bacterium, after obtaining the nutrient concentration at that bacterium's location.
- divideBacteria(): This method calls Bacteria.shouldDivide() and Bacteria.divide() to divide a bacterium and add the new daughter size to the media and array.
- computeSparseness(): This method determines how sparsely bacteria occupy the grid by comparing the number of cells to the occupied area.
- computeRoughness(): This method computes the distances from the colony centre to each bacterium and calculates the mean radius. The normalized standard deviation of these values is then computed.
- intermediateOutput(): This method calls detectClusters() and Medium.toString() to compile the state of every particle and cluster in the simulation at this point in time, and then writes it to the output file.



- startSimulation(): this method runs the entire simulation once the data has all been initialized. It writes the input parameters to the output file (which are needed for graphing). Then it adds the first bacterium to the medium and calls intermediateOutput() to output the initial state of the simulation. Then it enters a loop: timeStep() is called, then the number of ticks is increased by 1. This repeats for a set number of ticks decided by the config file, and then intermediateOutput() is called to save the current state to the output file, every time this happens is called 1 timepoint. This cycle continues until the simulation reaches the maximum amount of particles (maxAgents) specified in the config file and then the program stops. Alternatively, if 500 intermediateOutputs are produced then the program also stops.

- main(): This method in order parses the input and output file names, clears the output file, and then instantiates the simulation. In instantiating the simulation it reads the input file, initializes the medium, and opens the output file for writing. Finally, it starts the simulation.

Medium.java

- calculateMovements(): This calculates the force upon each particle. It traverses the whole particle array and calls calculateFNet() for each of them. It runs this in parallel using a forkJoin framework which uses TickStepAction objects.
- clearCoordinates(): This removes all particles from all coordinates in the grid so they can be placed into new grid squares once they move. It traverses the whole particle array and calls removeParticlesHere() for each of the MediaCoordinates that contains a particle, as this is easier than removing individual particles from MediaCoordinates one by one. It runs this in parallel using a forkJoin framework which uses TickStepAction objects.
- moveParticles(): This changes the position of every particle and then places it into a new MediaCoordinate on the grid to reflect the change in position. It traverses the whole particle array and calls move() for each of them. It runs this in parallel using a forkJoin framework which uses TickStepAction objects.
- produceEPS(): This checks if it is time for a Bacteria to produce EPS, and if so this Bacteria's timer is reset, and a new EPS is created 1 unit away from the Bacteria. It traverses the whole particle array and calls produceEPS() for all the bacteria, and ignores all the EPS. It runs this in parallel using a forkJoin framework which uses TickStepAction objects.
- detectClusters(): This method assigns all bacteria and EPS into clusters. Clusters are calculated separately for each particle type. A cluster is an isolated group of particles, where every member of the group is at least a minimum distance from any other particle of the same type that is not part of the cluster.
- diffuseAndConsume(): This method is responsible for the nutrient concentration being updated at each step. It utilises a forkJoin framework to implement nutrient diffusion through the DiffuseTask inner class. The method then implements nutrient consumption for each cell where bacteria are present, and then sets the new nutrients. The DiffuseTask inner class consists of the compute() and computeDirectly() methods. compute() calls computeDirectly() after dividing the grid into separate sections, and computeDirectly() implements diffusion.

Particle.java

- move(): This recalculates the position based on the forces acting upon it, and then replaces the particle into a MediaCoordinate based on its position.
- calculateRepulsiveForces(): This method gets a list of all particles close enough to possibly be overlapping with this particle. It then checks how far away each nearby particle is, and if they are overlapping then both particles experience a repulsive force.

Bacteria.java

- produceEPS(): This method decides whether this bacteria should produce an EPS on this tick. A counter counts down every tick, and if it reaches 0 then an EPS is produced and the timer is reset, so long as a minimum bacteria density is met and a maximum EPS density is not exceeded.
- calculateFNet(): This sums the random force, motility force, and repulsive forces experienced by the bacterium.

-grow(): This method uses various static parameters to calculate a differential equation and updates the length of the bacterium.

-shouldDivide(): This method uses an exponential function to determine a threshold value for whether the bacterium should divide after it reaches the maximum length, and returns either True or False.

-divide(): This method divides the bacterium and creates a new daughter cell with a new id and a slightly changed position. The original cell is reduced in size.

EPS.java

- calculateFNet(): This sums the random force, and repulsive forces experienced by the EPS.

ForceVector.java

- zeroVector: returns a zero vector: $\langle 0, 0 \rangle$

- multiply: returns a ForceVector equal to the calling vector with both components multiplied by the given scalar

- add: returns a ForceVector equal to the calling vector with both components added to by the given ForceVector's corresponding components

- getMagnitude: returns the magnitude of this ForceVector

- toUnitVector: returns a unit vector pointing in the same direction as the calling ForceVector.

TickStepAction.java

- compute(): this method is called by the methods in Medium which reference this. It does the forking and joining of the forkJoin framework and also actually does the functionality listed in those methods, depending on which one calls this.

Visualization.java

- readSimulationFile(): accepts a file name as input, it then uses the file data to set the values of the simulation parameters from the initial lines of the file and uses these to display key information on the left-hand side of the display. Thereafter it runs a loop wherein it processes each time point data, it reads the data, sets some parameters such as time point which are also displayed to the user, on the right-hand side of the display and then begins an inner loop which populates the object arrays with the data of the given time point. Once all the data for the given time point has been processed, it calls the repaint () (paintComponent) and then sleeps for the time configured by the user (if the manual click through is not set) or waits until the user continues the program (in the case of pause or manual click through).

- paintComponent(): overrides the JPanel paint method, to allow for the program to draw the bacteria and eps particles. It does this by calculating the size conversion needed based on the frame size and the simulation boundary, and then by iterating over the ArrayLists of objects and drawing them onto the main panel.

- toScreen(): converts simulation x and y points to display points.

- `generateColor()`: uses the cluster ID and size to generate a colour for the display, such that small clusters are darker and larger clusters are brighter, but all eps or bacteria are the same colour. This is set to blue for bacteria and red for EPS.
- `showOptionsDialog()`: displays all the visualization settings to the user, acts as a small user interface for the user to enter the input file name, the time delay between frames and set the other preferences for each visualization. It then checks that the data entered is valid (if not, it asks the user to retry). It then sets the appropriate visualisation parameters and returns to the calling method.
- `startPlayback()`: a small method to start a new thread which controls reading in the file and signalling the display to update at the appropriate intervals based on the user settings.
- `main()`: controls the main elements of the visualisation, it begins by creating a visualization object and calling the `showOptionsDialog()` method, thereafter (if the method returns true) it adds a frame object with some control buttons and overall pieces of the display. Once the display is set up and becomes visible, the action listener on the frame triggers the `startPlayback()` method, which begins the reading of the file.

GraphingModule.py

Data Structures

The Graphing Module is a Tkinter-based GUI tool that loads simulation output data, organises into structured dictionaries and lists, and uses these data structures to plot various metrics.

The main data structure is `self.simulations`. This is a dictionary that maps each file name to its simulation data. Each simulation data is a list of time points that contains the metadata for each timepoint, as well as the lists of Bacteria, EPS, Bacteria Clusters, and EPS Clusters.

Important Methods

- `load_simulations()` and `load_simulation_data()` are responsible for handling the data, and stores it in `self.simulations` after converting it to the correct structure.
- `plot_eps_clusters()` groups simulations by their depletion effects and motility forces, and plots the largest 50 EPS clusters at the final timepoint.
- `plot_eps_phase_diagram()` returns a scatterplot of depletion vs motility, where each point is a cluster that is colour coded by its size.
- `plot_bacterial_clusters()` is similar to the `plot_eps_clusters()` method, but plots the largest bacterial clusters instead.
- `plot_roughness_evolution()` plots the change in roughness of the colony over time.
- `plot_colony_morphology()` plots the roughness and sparseness evolution over time for multiple simulation outputs with different initial concentrations.

Special class relationships:

As stated in the design section UML diagram, Particle is an abstract parent class of Bacteria, and EPS as this reduced code duplication a lot.

Medium stores many bacteria and EPS, but they also have a link back to the medium they are part of as it provides a significantly simpler implementation of methods that involve detecting nearby particles and processing data within a particle. Primary examples are calculating repulsive forces.

Finally, Medium makes use of the TickStepAction class to define work for running code within a forkJoin framework.

For the visualization module, the class relationships are as per the UML diagram. The visualisation class stores ArrayLists which contain multiple bacteria objects, eps objects and cluster objects. Each bacteria and eps object, while not directly linked to a cluster object, is associated with a given cluster, this association is used by the generateColor() method as it uses the clusterID from the bacteria or eps to reference the cluster array and find the size of the cluster, which is then used to calculate the brightness of the cluster.

Noteworthy libraries used:

Java's forkJoin framework was used to multithread the methods calculateMovements, clearCoordinates, moveParticles, and produceEPS.

For the graphing module, Python's NumPy library was used for data loading and storage and Matplotlib and threading were used for graphing. Tkinter was used for the GUI.

Program Validation and Verification:

Input validation:

The simulation validates the config file entered, to check that numbers that need to be integers are integers, floats are floats, empty lines are where they should be, and numbers that must be positive numbers are positive. If any of the input format is not adhered to, it sends an error message telling the user which input field is wrong, and why it is wrong, and then closes the program without running the simulation. No stack traces are shown when such an error is made.

Here is an example where the config file had the maxAgents field replaced with a string:

```
1 0, 0
2
3 hello!|
4 0.05
5
6 500
7 4
8 4
9 300
10
11 0.5
12 20000
13
Running simulation with:
Input File = simulationConfig.txt
Output File = simulation_output.txt
Invalid value for maxAgents: 'hello!'
This should be an integer
Program terminating
PS C:\Users\Bynac\OneDrive - University of Cape Town\UCT\CSC3003S\biofilm-main> |
```

When using the makefile, if the user enters an invalid file name the program sends a similar message.

The visualization module performs a small amount of input validation when the visualization settings are entered; it verifies that the file name entered is a valid file as well as checking that the value of frame delay is a positive integer and the values of the EPS and bacteria scaling factors are valid floats.

The visualization module does not verify the content and structure of the input file because this is expected to be taken directly from the simulation module and thus is expected to be of the correct format with acceptable values.

Testing and verification:

process	technique
1. Class Testing: test methods and state behaviour of classes	Random, and Unit tests
2. Integration Testing: test the interaction of sets of classes	Random, and Unit tests
3. System Testing: test the behaviour of the whole system, and with interactions between modules	Random, Behavioural, and Performance tests, including Mocking

The testing plan for the program is as detailed above. This program uses a visualization paired with the simulation, and the original paper we are trying to reproduce has videos of their visualization. These serve as a benchmark for us to use to reference whether our system behaves correctly. The visualization is a key aspect as it allows us to see the behaviour of the simulation in action, and if the visualization itself is misbehaving it is visually obvious.

The primary testing method used was unit testing. Most methods were tested using unit tests. Some were left out as their functionality is trivial: Get, and Set methods weren't tested, due to being simple enough to assume safe, as well as toString methods since they directly can be seen as dysfunctional during output if they don't work. Constructors that merely set instance

variables or similar were also considered safe. The compute() method of TickStepAction was tested within the Unit tests of Medium and therefore did not explicitly receive its own tests.

Simulation verification:

Methods of the simulation were unit tested bottom up. Tests were not exhaustive of all possibilities since that is physically impossible. Tests covered all key aspects of the codes functionality but were not more detailed than necessary to consider the code correct. Simpler methods were tested first, then more complicated ones. Some unit tests relied on previous ones being successfully tested, since some parts of the system are too interlinked to viably isolate them for tests. This is ok since the simpler methods all passed their tests before the more complicated methods started using them in their tests, hence this is considered safe. Due to the Simulation class primarily calling other methods rather than doing calculations, the following methods were not Unit tested as it is possible to see errors in them quite easily using the visualization or by looking at the output file and seeing incorrect formatting or results: the constructor, startSimulation(), timeStep(), IntermediateOutput(), and main(). MediaCoordinate has no Unit tests, as all of its methods are essentially get and set methods.

Results of the unit tests are appended in appendix B.

Behavioural tests: the system was tested alongside the visualization many times, and the output files it produced were reviewed on many occasions and they followed the correct output format, and expectations, and therefore was considered correct.

The system was also tested with instrumentation for detecting performance of the different algorithms. These results are summarized in appendix B as well.

Visualization verification:

Methods of the visualization class were tested minorly using unit tests to verify correct calculations or conversions as well as consistency. However, due to the nature of the visualization module, most of the testing consisted of usage tests, involving various input files and observations of the results as the main output of the visualization is visually obvious in case of errors. Further testing involving the user settings was performed in a similar manner, initially each setting was tested individually to ensure it functioned correctly in isolation, thereafter several combinations of possible settings were tested to ensure all settings continued to behave as expected when used simultaneously.

Overall system integration tests were continually performed thereafter as the visualization module was used to verify output from the simulation module and refine properties thereof.

Performance for the visualization module were not tested with any instrumentation; however it always runs smoothly and displays the simulation quickly even under short frame delays.

Conclusion:

The aim of this project was to develop a particle-based biophysical model that simulates early-stage biofilm formation, with focus on the role of extracellular polymeric substances (EPS) in driving cell aggregation. The model successfully represents a growing and expanding bacterial colony of mobile, bacteria which move, reproduce, and secrete EPS. The

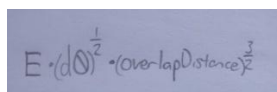
simulation is a multithreaded Java application. It has a simulation module, visualization module for presentation of the results, and a graphing module to present graphs of the results.

Aims Met

Simulation

The user can use a makefile to select the input and output files of the simulation. The simulation also validates the input file and tells the user what they did wrong, if any input is invalid, without throwing stack traces.

The simulation runs according to the descriptions given in Bera et al. (2023). Bacteria grow along their axes and may divide after reaching a certain length. Their growth depends on nutrients in the surrounding medium. Bacteria also periodically produce EPS, given that there is enough bacteria, and few enough EPS in the surrounding area. Finally, all particles experience overdamped mechanics forces, where there is no momentum. Bacteria are self-propelling according to their motility, and they propel themselves in a random direction. All particles experience a random force of 0 to 10^{-3} at each simulation tick. All particles are also repulsed by overlapping particles, according to the formula in the paper:


$$E = (d_0)^{\frac{1}{2}} \cdot (\text{overlapDistance})^{\frac{3}{2}}$$

Here, E is the elastic modulus, which is a constant, selected by the user in the input file. It is different for the interaction between EPS and EPS, EPS and Bacteria, and Bacteria and Bacteria. d_0 is the radius of the particle, and overlapDistance , is the distance of overlap between the particles.

All these mechanisms were successfully unit tested and followed the formulas given in the paper as expected. Those test results are found below in Appendix B, and in the codebase.

Despite following the descriptions in the paper, the simulations' results are not the same. Particles jump about and move much quicker than in the videos in Bera et al. (2023). Our Bacteria also grow faster, and particles form into groups less than in their paper. It is not possible to explain these disparities with certainty.

Possible explanations are that our bacteria do not experience torque properly and are not rotated when repulsed by nearby particles. This is something that could be improved on without further knowledge of the paper. It's also possible that since our growth appears to be faster than theirs, particles do not have time to move around the way theirs did. Our growth mimics the paper's description so this is an unexpected result.

Visualization

In our visualization, the user can select many options at the menu, and they are well described and easy to use. The video-like display of the simulation shows all the particle positions over time. Clusters can also be shown and appear as lighter colours the larger they are. The performance of the visualization's playback also far outdoes the simulation, so it doesn't take as long to view the simulation multiple times.

Some extra features include the ability to pause or resume the display, and the ability to rewatch the simulation. Once completed the user can return to the settings menu.

Settings include: the ability to adjust the frame rate (done by reducing the delay between the frames). Either Bacteria or EPS can be hidden to make the other more obvious, and their colours can be changed too. This especially helps when viewing clusters. The background colour can also be changed. The user can also choose to manually click through frames instead of them going by automatically. The user can also inflate the size of EPS or Bacteria on screen to make them easier to see which is especially helpful if you have a big simulation boundary. Finally, the user can exit at any time without any errors.

Graph Results

Running the simulation module results in the generation of an output file that is used to produce a visual playback of colony growth with the visualization, as well as various graphs produced in the paper. These graphs are as follows:

This graph represents Figure 3 in the paper, and similarly demonstrates the first 50 EPS cluster sizes, which demonstrate suppression of clustering at moderate to high depletion.

The phase diagram representing Figure 4 also demonstrates increasing cluster size as the depletion effect increases for a given motility force, demonstrating a tendency towards phase separation.

Similarly to the EPS Cluster Size graph, the Bacteria cluster size graph representing Figure 6 demonstrated a suppression of clustering at moderate to high depletion.

The temporal evolution of roughness did not have as linear a relationship as Figure 8 of the paper.

Similarly, Figure 11 of the paper was not replicated as well by the plots of roughness and sparseness against time.

Summary

Overall, the system is robust, and well modularised, making it more maintainable. It also has a series of unit tests to aid in maintaining the code were it ever necessary. The system is a successful simulation that models the problem, but sadly it could not replicate Bera et al. (2023)'s results, and we are not sure exactly why. Likely there are details in their implementation which were not described in sufficient detail to replicate their results.

References:

Bera, P. (2023) Interplay of cell motility and self-secreted extracellular polymeric substance induced depletion effects on spatial patterning in a growing microbial colony. *Soft Matter*, 19, 8136-8149.

*The paper above is referred to in the report as 'Bera et al. (2023)' and the paper below as 'Bera et al. (2022)'

Bera, P. (2022) A mechanistic understanding of microcolony morphogenesis: coexistence of mobile and sessile aggregates. *Soft Matter*, 19, 1034-1045.

-Appendices-

Appendix A: User manual

Makefile:

There is a makefile that can run any of the 3 modules. The commands for each module are described in their separate sections.

Using the Simulation

The simulation is the main set of calculations within the program. This Simulates the growth of a bacterial colony with a biofilm starting with a single bacterium inoculated at the centre of the simulation grid.

To run the simulation, you need a configuration file. A sample file: “SimulationConfig.txt” is given.

There are 3 ways to run the simulation:

- Makefile (recommended method): run ‘make all’ to compile all files, then
Use the command format
`<make run ARGS="simulationConfig.txt simulation_output.txt">`
Swap out simulationConfig.txt for your input file, and simulation_output.txt for your output file. You can also just use “make run” which will use the 2 files stated above as defaults.
- Command line: after running ‘make’ you can use
“java -cp bin Simulation.Simulation” instead of the makefile. This method does not let you choose your input and output files, however.
- IDE (not recommended): If you open the simulation in your IDE, make sure your terminal is in the ‘biofilm-main’ directory. Then you can run the main method of Simulation.java, and it will use simulationConfig.txt as its input, so you will need to edit that if you use this method.

The full file format is below. The only simulation parameters worth changing in a practical scenario are: maxAgents, Motility, and the ElasticModulus. Others can be changed for testing purposes when using the program too.

Here is the format, and an example (This is also in the files with the code):

Format:

(the words before a ‘#’ are the names for the values, the words after a ‘#’ are comments about sections of parameters or whether the number is an integer or float, and what the number represents. Note that sometimes the comments are too long for this document and flow onto the next line)

origin_x, origin_y #int, int: starting position of the first bacterium (recommended: 0, 0)
--

#simulation values

max_agents #int: max number of particles – the simulation stops once this is reached (recommended: 10000 - 100000)
output_interval #int: how often (in in-simulation hours) output is sent to the output file (recommended: 0.1)

#medium values

simulation_boundary #int, distance from (0;0) to the edge of the simulation grid – make this big enough that no particle reaches the edge ideally (recommended: 100-1000)
initial_nutrient_concentration #float, starting nutrients per μm^2 . (recommended: ~ 10)
consumption_rate #float, rate that a bacteria consumes nutrients (recommended: ~ 4)
diffusion_rate #float, rate that nutrients diffuse to nearby gridsquares (recommended: ~ 300)

#EPS values

EPS_size #float, diameter of EPS in μm (recommended: 0.5 - 1)
EPS_EPS_Elastic_Modulus #float, constant that increases repulsive forces between 2 EPS, in Pa (recommended: 20000 - 70000)

#bacteria values

bacteria_lmax #float, maximum length of a bacterial cell in μm (recommended: 5)
bacteria_diameter #float, diameter of bacteria in μm (recommended: 1)
bacteria_motility #float, motility force of bacteria in $\text{Pa} \cdot \mu\text{m}^2$ (recommended: 0 - 1100)
bacteria_growth_rate #float, bacteria growth rate in $\mu\text{m}/\text{hr}$ (recommended: ~ 3.5)
bacteria_reproduction_rate #float, bacteria reproduction rate, in bacteria/hr (recommended: 1)
bacteria_EPS_production_rate #float, EPS production rate, in EPS/hr (recommended: 1)
Bacteria_EPS_Elastic_Modulus #float, constant that increases repulsive forces between an EPS and a Bacteria, in Pa (recommended: 20000 - 70000)

Bacteria_Bacteria_Elastic_Modulus #float, constant that increases repulsive forces between 2 Bacteria, in Pa (recommended: 20000 - 70000)
Friction #float, constant that decreases all movement (recommended: 200)
EPS_Production_Min_Cell_Density #minimum bacteria density required to produce EPS, in μm^2 of bacteria/ μm^2 (recommended: 5)
EPS_Production_Max_EPS_Density #maximum eps density before bacteria stop producing eps, in μm^2 of eps/ μm^2 (recommended: 0.3)

Example:

0, 0

1000
0.1

500
10
4
300

```
0.5
40000

5
1
100
3.5
1
1
40000
70000
200
5
0.3
```

Using the graphing module

The graphing module allows the user to input the names of multiple simulation output files and reproduces figure from the paper using these files. Similarly to the simulation module, there are 3 ways to run the simulation:

- Makefile (recommended): run “make all” to compile all files, then use the command “make graphing” to start the Graphing Module.
- Command Line: python3 GraphingModule.py
- IDE (not recommended): run GraphingModule.py in your IDE terminal, ensuring that the terminal is in the “biofilm-main” directory. If your desired input files are in the same directory, the file will run as with the makefile.

After starting the graphing module, you are given the option to upload your files by manually typing the file names into a text box, or by browsing through and selecting files from your file browser. After the files have been selected, pressing “Load Simulations” button will read in the data from each file. After this, you are able to select from five possible graphical outputs.

The input files are the files generated by the simulation module, and take on the same format as specified above. Note that the graphing module works best with multiple simulation output files where various parameters in the config file have been changed.

Using the visualization module

The visualization is the section of the program which allows users to view the results of a given simulation. This provides a visual depiction of the simulated growth of the bacterial colony, as generated in the simulation module. When run, it allows the user to adjust several parameters which allow the user to view the simulation as they desire, to highlight key aspects of the simulation if needed.

To run the visualization, you need a configuration file in the same format as is produced by the Simulation module. A sample file: “SimulationOutput.txt” is given.

There are 3 ways to run the visualization:

- Makefile (recommended method): run ‘make all’ to compile all files, then

Use the command “make visualization” which will start the module and direct you to the options menu

- Command line
- IDE: If you open the visualization in your IDE, make sure your terminal is in the ‘biofilm-main’ directory. Then you can run the main method of Visualization.java, and it will run as with the Makefile, giving a list of settings to adjust before viewing the simulation results.

Here is the format, and an example:

Format:

```
#simulation values
max_agents #int: max number of particles – the simulation stops once this is reached
output_interval #int: how often (in in-simulation hours) output is sent to the output file

#EPS values
EPS_size #float, diameter of EPS in um
EPS_EPS_Elastic_Modulus #float, constant that increases repulsive forces between 2 EPS,
in Pa

#bacteria values
bacteria_lmax #float, maximum length of a bacterial cell in um
bacteria_diameter #float, diameter of bacteria in um
bacteria_motility #float, motility force of bacteria in Pa*um^2
bacteria_growth_rate #float, bacteria growth rate in um/hr
bacteria_reproduction_rate #float, bacteria reproduction rate, in bacteria/hr
bacteria_EPS_production_rate #float, EPS production rate, in EPS/hr
Bacteria_EPS_Elastic_Modulus #float, constant that increases repulsive forces between an
EPS and a Bacteria, in Pa
Bacteria_Bacteria_Elastic_Modulus #float, constant that increases repulsive forces
between 2 Bacteria, in Pa
Friction #float, constant that decreases all movement
EPS_Production_Min_Cell_Density #minimum bacteria density required to produce EPS,
in bacteria/um^2
EPS_Production_Max_EPS_Density #maximum eps density before bacteria stop
producing eps, in eps/um^2

#medium values
simulation_boundary #int, distance from (0;0) to the edge of the simulation grid – make
this big enough that no particle reaches the edge ideally
initial_nutrient_concentration #float, starting nutrients per um^2.
consumption_rate #float, rate that a bacteria consumes nutrients
diffusion_rate #float, rate that nutrients diffuse to nearby gridsquares

#Output at each time point
time_point #int, the current time point for which the following data applies
ticks_since_start #int the time in ticks (360*simulation hours) since the simulation began
colony_centreX, colony_centreY #int, int
```

colony_diameter #float
roughness #float
sparseness #float
Bacteria: #Title to indicate the start of the Bacteria details and to increase readability
ID, x, y, xFacing, yFacing, length, clusterNumber #For each bacterium (int, int, int, float, float, float, int)
EPS: #Title to indicate the start of the EPS details
ID, x, y, clusterNumber #For each eps (int, int, int, int)
Bacterial Clusters: #Title to indicate the start of the Bacteria cluster details
ID, numberOfParticles #For each cluster (int, int)
EPS Clusters: #Title to indicate the start of the EPS cluster details
ID, numberOfParticles #For each cluster (int, int)

Example:

10000
0.1

200
3.0
4.0
300.0

0.5
40000.0

5.0
1.0
500.0
3.5
1.0
360.0
40000.0
70000.0
200.0
5.0
0.3

0
0
0.0, 0.0
1.0
0.0
0.0
Bacteria:
0, 0, 0, -0.6628665521644692, 0.7487375601781903, 1.0, 0
EPS:
Bacterial Clusters:
0, 1
EPS Clusters:

1
36
-2.0, 2.0
4.444131728146501
0.0
0.0
Bacteria:
0, -2, 2, -0.6628665521644692, 0.7487375601781903, 4.444131728146501, 0
EPS:
0, 0, 0, 0
Bacterial Clusters:
0, 1
EPS Clusters:
0, 1

2
72
-3.0, 3.0
7.594131728146498
0.0
0.0
Bacteria:
0, -3, 3, -0.6628665521644692, 0.7487375601781903, 7.594131728146498, 0
EPS:
0, 0, 0, 0
Bacterial Clusters:
0, 1
EPS Clusters:
0, 1

3
108
-3.0, 3.0
7.1718264127284606
0.0
0.9953379953379954
Bacteria:
0, -74, 4, -0.6628665521644692, 0.7487375601781903, 7.256993321505826, 0
1, 68, 2, 0.6869828175375697, -0.7266736601860164, 7.086659503951095, 0
EPS:
0, 0, 0, 0
1, -1, 3, 0
Bacterial Clusters:
0, 1
1, 1
EPS Clusters:
0, 2

Appendix B: Test results

Unit tests:

All unit tests are available in the code within the folder 'lib.' Otherwise here is a summary screenshot of the tests being successful for half (movement and EPS production) of the simulation.

✓ 1/1

1.3s

✓ biofilm-main 550ms

✓ { } lib 550ms

✓ BacteriaTest 103ms

- ✓ produceEPSshouldproduceEPSwhenconditionsAreRight() 23ms
- ✓ produceEPSwaitingWorksCorrectly() 3.0ms
- ✓ produceEPSneedsCellToBeLong() 0.0ms
- ✓ produceEPSneedsCellToBeLongOrMoreNearby() 0.0ms
- ✓ produceEPSshouldStopIfTooManyNearbyEPS() 5.0ms
- ✓ produceEPSshouldStopAtASpecificAmountOfEPSNearby() 7.0ms
- ✓ calcFNetShouldSumOtherForces() 24ms
- ✓ calcFNetShouldSumOtherForces2() 22ms
- ✓ motilityForceShouldReturnItself() 8.0ms
- ✓ motilityForceSplitsAlongAxesCorrectly() 1.0ms
- ✓ elasticModulusWithBacteria() 1.0ms
- ✓ elasticModulusWithEPS() 0.0ms
- ✓ isTypeEPS() 0.0ms
- ✓ isTypeBacteria() 9.0ms

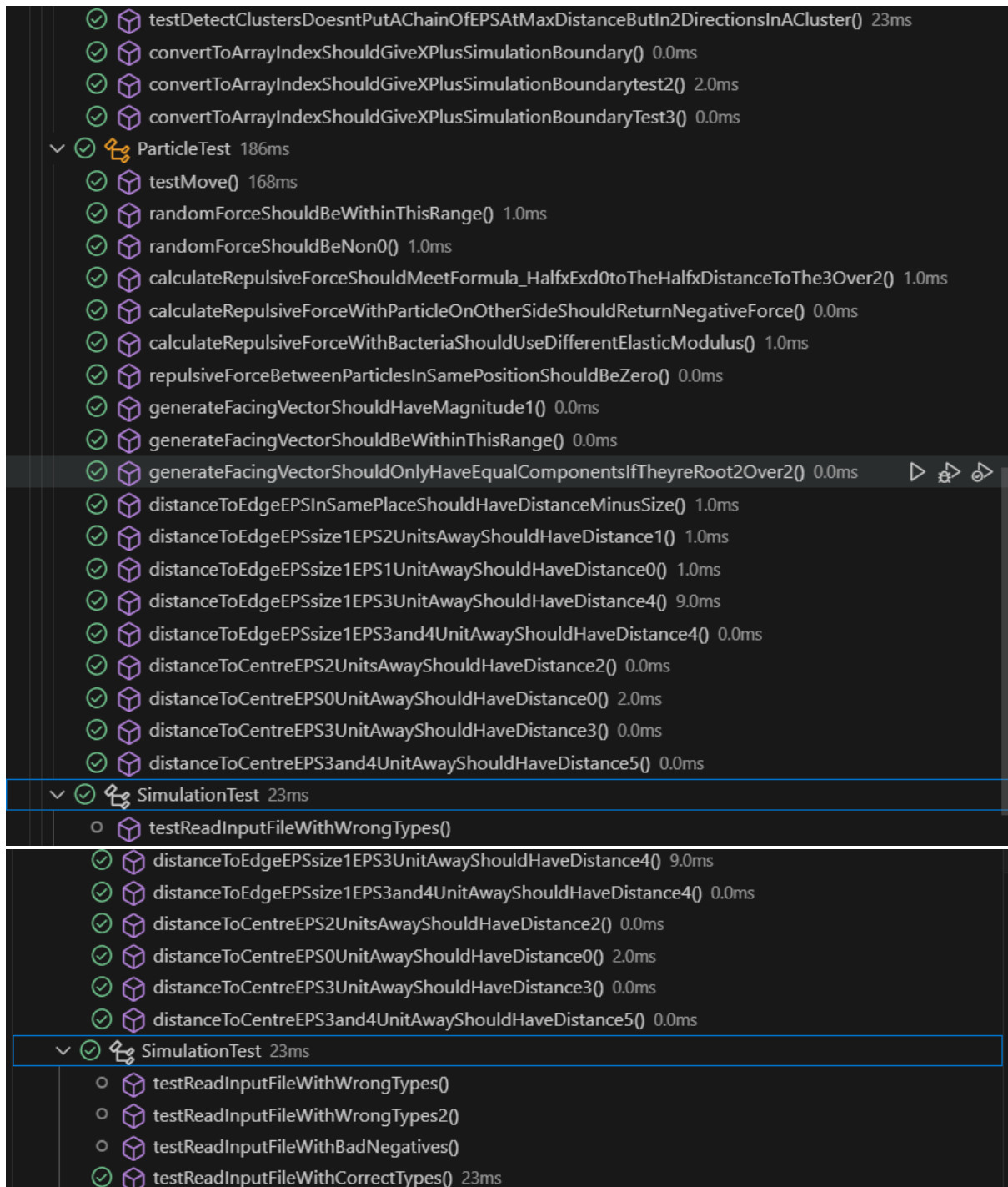
✓ EPSTest 20ms

- ✓ elasticModulusWithBacteria() 10ms
- ✓ elasticModulusWithEPS() 1.0ms
- ✓ isTypeEPS()
- ✓ isTypeBacteria() 9.0ms

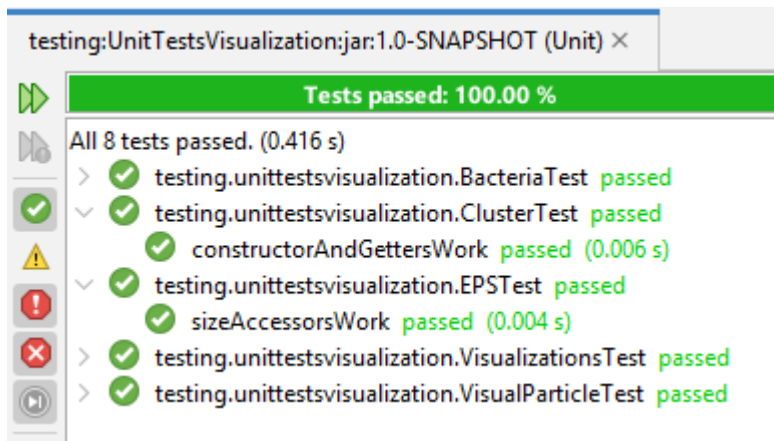
✓ ForceVectorTest 17ms

- ✓ TestZeroVector() 1.0ms
- ✓ multiplyByZeroShouldReturnZeroVector() 1.0ms
- ✓ multiplyByPositiveScalarShouldScaleBothComponents() 0.0ms
- ✓ multiplyByNegativeScalarShouldFlipDirection() 1.0ms
- ✓ multiplyByOneShouldReturnSameVector() 1.0ms
- testAdd(double, double, double, double, double, double)
- ✓ magnitudeOfZeroVectorShouldEqualZero() 0.0ms

- ✓  magnitudeOfZeroVectorShouldEqualZero() 0.0ms
- ✓  magnitudeOfPositiveVectorShouldBePositive() 0.0ms
- ✓  magnitudeOfNegativeVectorShouldBePositive() 1.0ms
- ✓  unitVectorOfZeroVectorIsZeroVector()
- ✓  magnitudeOfUnitVectorShouldBeOneXOnly() 0.0ms
- ✓  magnitudeOfUnitVectorShouldBeOneYOnly() 1.0ms
- ✓  magnitudeOfUnitVectorShouldBeOne() 11ms
- ✓  MediumTest 201ms
 - ✓  constructorTest() 31ms
 - ✓  testAddBacteria() 3.0ms
 - ✓  testAddEPS() 0.0ms
 - ✓  testPlaceParticle() 8.0ms
 - ✓  testPlaceParticlePutsParticleInCorrectSpot() 18ms
 - ✓  testClearCoords() 6.0ms
 - ✓  testMoveParticlesMovesThings() 2.0ms
 - ✓  testProduceEPSshouldproduceEPSsometimes() 16ms
 - ✓  testGetNearbyParticlesWithParticlesAtEdgeOfSearch() 0.0ms
 - ✓  testGetNearbyParticlesWithParticlesAtOtherEdgeOfSearch() 0.0ms
 - ✓  testGetNearbyParticlesWithParticlesOutsideEdgeOfSearch() 7.0ms
 - ✓  testCountNearbyParticlesWithParticlesAtEdgeOfSearch() 4.0ms
 - ✓  testCountNearbyParticlesWithParticlesAtOtherEdgeOfSearch() 4.0ms
 - ✓  testCountNearbyParticlesWithParticlesOutsideEdgeOfSearch() 3.0ms
 - ✓  testCalcMovementsUpdatesAllParticlesFNet() 1.0ms
 - ✓  testDetectClustersPutsAChainOfEPSAtMaxClusterDistanceinOneCluster() 63ms
 - ✓  testDetectClustersPutsAChainOfEPSAtMaxClusterDistanceinOneClusterButNegativeCoords() 2.0ms
 - ✓  testDetectClustersDoesntPutAChainOfEPSPastMaxClusterDistanceinOneCluster() 8.0ms
 - ✓  testDetectClustersDoesntPutAChainOfEPSAtMaxDistanceButIn2DirectionsInACluster() 23ms



The visualization unit test results are as follows:



The graphing unit test results are as follows:

```
sonalinalidoo@Sonalis-MacBook-Air biofilm % python3 -m unittest -v TestGraphingModule.py
test_clear_all_resets_state (TestGraphingModule.TestGraphingModule.test_clear_all_resets_state) ... ok
test_get_debug_info_returns_string (TestGraphingModule.TestGraphingModule.test_get_debug_info_returns_string) ... ok
test_load_simulation_data (TestGraphingModule.TestGraphingModule.test_load_simulation_data) ... ok
test_plot_eps_clusters_runs_without_error (TestGraphingModule.TestGraphingModule.test_plot_eps_clusters_runs_without_error) ... Processing depletion 10-5, motility 1
EPS clusters found: 1
Plotting 1 points
ok
-----
Ran 4 tests in 0.731s
OK
```

Performance tests:

The simulation was instrumented in a few ways. To monitor the performance of different aspects of the system.

First the seven functions that happen at every timestep had their total time recorded, and displayed in order: 1, time to calculate FNet for all particles. 2, time to clear all coordinates of particles. 3, time to move particles and place them onto new coordinates. 4 time to grow bacteria. 5, time to divide Bacteria. 6, time to diffuse nutrients. 7, time to produce EPS. This is an example of the program having run:

```
Timepoint: 44
Timepoint: 45
total time: 45988ms
212 82 110 27140 16836 1258 202
PS C:\Users\Bynac\OneDrive - Unive
```

Here we see the bottlenecks are bacterial growth, and division. This took around 45s to get to 1000 particles.

Next the time it takes to calculate all the clusters and produce an intermediate output to the output file was compared to the rest of the calculations. This test showed that intermediate output is not a bottleneck.


```
time between outputs: 977 ms, time steps: 973 ms, intermediate output: 3395 us  
Timepoint: 39  
time between outputs: 953 ms, time steps: 948 ms, intermediate output: 5114 us  
Timepoint: 40  
time between outputs: 995 ms, time steps: 990 ms, intermediate output: 4656 us  
Timepoint: 41  
time between outputs: 991 ms, time steps: 986 ms, intermediate output: 5145 us  
Timepoint: 42  
time between outputs: 964 ms, time steps: 957 ms, intermediate output: 7228 us  
Timepoint: 43  
time between outputs: 1060 ms, time steps: 1050 ms, intermediate output: 9307 us  
Timepoint: 44  
time between outputs: 989 ms, time steps: 976 ms, intermediate output: 12612 us
```

Each intermediate output was instrumented. Note the times for intermediate output are in microseconds, while the rest is in milliseconds.

Appendix C: How work was delegated between members:

Stage 1 (risk, roles and scope):

Risk assessment – Byron

Roles and responsibility delegation – Decided by all members

Scope – Mostly Ruth, and Sonali, some help from Byron

Stage 2 (diagrams):

Use case descriptions – Ruth

Analysis Class model - Byron

Interaction Diagram or State Diagram – Byron

Project plan – Ruth

Preliminary test plan - Sonali

Stage 3 and Implementation phase:

Simulation module:

Byron created the simulation module, and coded functionality for all movement calculations, and EPS production, and clustering calculations. He also performed unit tests for these functions. Sonali coded bacterial growth and division mechanics, and nutrient diffusion and consumption. She also performed unit tests for these functions.

Visualization module:

All coding in this module was done by Ruth.

Graphing module:

All coding in this module was done by Sonali.

Compatibility, Input, and output files:

Byron designed the simulation config, and output files. All 3 members helped refine the details of these files.

Skeleton code and GitLab setup:

Ruth created all classes and methods from the original UML diagrams. She also created the GitLab and added all team members.

Report: Byron started the report and fitted the specified template from Amathuba. The Abstract was done by Ruth. The introduction was done by Byron. Ruth did the requirements section. Byron did the overall design explanation, Simulation UML diagram, and explanation of the methods he coded, similarly Ruth did the Visualization UML diagram and explanation of the visualization code. Byron explained the implementation of the parts he coded, Ruth did the same for the visualization module. Sonali did the explanation of the simulation methods she coded, as well as the graphing explanation. Byron made the test cases table and did Unit tests for his code. Ruth performed unit tests, UI tests and overall output testing for the visualization module. Sonali performed unit tests and UI tests for the graphing module, as well as wrote unit tests for the simulation module. Conclusion was written by Sonali.

In appendix A, Byron did the Simulation, Ruth did the Visualization, and Sonali did the graphing module. Appendix B was done by all 3 members, with each doing the Unit tests for their own section, and the Simulation performance tests were done by Byron. Appendix C, and D were started by Byron and expanded by Ruth and Sonali.

Appendix D: Declaration of generative AI uses

Simulation module:

Config file parsing method was created by ChatGPT and edited by Byron.

The FakeSimulationDataGenerator class was created fully by ChatGPT

Code for placeParticle method in Medium.java was created with help from Claude

Bugfixing and optimisation of detectClusters method was done with help from Claude

The updated UML diagram for the simulation was made with help from ChatGPT

The bacterial growth and division and the nutrient diffusion and consumption methods were debugged with the aid of ChatGPT.

The readSimulationFile method was created by ChatGPT and edited by Ruth to fit the updated file format as well as include the JLabel features.

The generateColor method was created by ChatGPT and edited by Ruth.

Editing and bug fixing of the PaintComponent method was done with assistance from ChatGPT.

Claude was used to generate a framework for the Graphing Module.